

MongoDB Aggregation – Interview Revision Guide

Syntax Cheat Sheet

\$match

```
{ $match: { status: "Delivered" } }
```

\$project

```
{ $project: { _id:0, customerName:"$customer", amount:1 } }
```

\$group

```
{ $group: { _id:"$customer", total:{ $sum:"$amount" }, count:{ $sum:1 } } }
```

\$cond

```
{ $cond:{ if:{ $gte:["$salary",80000] }, then:"Senior", else:"Junior" } }
```

\$switch

```
{ $switch:{ branches:[{ case:{ $gte:["$salary",100000] }, then:"Gold" }], default:"Bronze" } }
```

\$lookup

```
{ $lookup:{ from:"users", localField:"userId", foreignField:"_id", as:"userInfo" } }
```

\$lookup + let

```
{ $lookup:{ from:"orders", let:{ userId:"$_id" }, pipeline:[{ $match:{ $expr:{ $eq:["$userId","$$userId"] } } }], as:"orders" } }
```

\$filter

```
{ $filter:{ input:"$marks", as:"item", cond:{ $gte:["$$item.score",70] } } }
```

\$map

```
{ $map:{ input:"$marks", as:"item", in:"$$item.subject" } }
```

\$reduce

```
{ $reduce:{ input:"$marks", initialValue:0, in:{ $add:["$$value","$$this"] } } }
```

\$facet

```
{ $facet:{ total:[{ $group:{ _id:null, count:{ $sum:1 } } } ] } }
```

\$setWindowFields

```
{ $setWindowFields:{ sortBy:{ date:1 }, output:{ runningTotal:{ $sum:"$amount", window:{ documents:["unbounded","current"] } } } }
```

\$ifNull

```
{ $ifNull:["$phone","Not Available"] }
```

\$\$ROOT

```
{ $push:"$$ROOT" }
```

JavaScript Mapping: filter→\$filter | map→\$map | reduce→\$reduce | ??→\$ifNull | if/else→\$cond | Promise.all→\$facet | {...obj}→\$\$ROOT

Q1. Return only Delivered orders, rename customer → customerName, and hide _id.

Concepts Used: \$match • \$project

```
db.orders.aggregate([
  { $match: { status: "Delivered" } },
  {
    $project: {
      _id: 0,
      customerName: "$customer",
      amount: 1
    }
  }
])
```

Memory: Filter first, then reshape.

Q2. Return totalAmount and orderCount for each customer.

Concepts Used: \$match • \$group • \$sum • \$project

```
db.orders.aggregate([
  { $match: { status: "Delivered" } },
  {
    $group: {
      _id: "$customer",
      totalAmount: { $sum: "$amount" },
      orderCount: { $sum: 1 }
    }
  },
  {
    $project: {
      _id: 0,
      customerName: "$_id",
      totalAmount: 1,
      orderCount: 1
    }
  }
])
```

Memory: Every field inside \$group (except _id) needs an accumulator.

Q3. Return Top 2 customers by revenue.

Concepts Used: \$group • \$sort • \$limit

```
db.orders.aggregate([
  { $match: { status: "Delivered" } },
  {
    $group: {
      _id: "$customer",
      totalAmount: { $sum: "$amount" }
    }
  },
  { $sort: { totalAmount: -1 } },
  { $limit: 2 }
])
```

Memory: Sort uses field names.

Q4. Flatten skills array.

Concepts Used: \$unwind • \$project

```
db.employees.aggregate([
  { $unwind: "$skills" },
  {
    $project: {
      _id: 0,
      name: 1,
      skill: "$skills"
    }
  }
])
```

Memory: \$unwind creates one document per array element.

Q5. Join orders with users.

Concepts Used: \$lookup • \$unwind • \$project

```
db.orders.aggregate([
  {
    $lookup: {
      from: "users",
      localField: "userId",
      foreignField: "_id",
      as: "userInfo"
    }
  },
  { $unwind: "$userInfo" },
  {
    $project: {
      _id: 0,
      orderId: "$_id",
      name: "$userInfo.name",
      amount: 1
    }
  }
])
```

Memory: \$lookup returns an array.

Q6. Users with totalSpent.

Concepts Used: \$lookup • preserveNullAndEmptyArrays • \$group

```
db.users.aggregate([
  {
    $lookup: {
      from: "orders",
      localField: "_id",
      foreignField: "userId",
      as: "orderInfo"
    }
  },
  {
    $unwind: {
      path: "$orderInfo",
      preserveNullAndEmptyArrays: true
    }
  },
  {
    $group: {
      _id: "$name",
      totalSpent: {
        $sum: { $ifNull: ["$orderInfo.amount", 0] }
      },
      orderCount: {
```

```

    $sum: {
      $cond: [
        { $ifNull: ["$orderId", false] },
        1,
        0
      ]
    }
  }
}
}
})

```

Memory: Keeps left join behavior.

Q7. Return passed subjects.

Concepts Used: \$filter

```

db.students.aggregate([
  {
    $project: {
      name: 1,
      passedSubjects: {
        $filter: {
          input: "$marks",
          as: "item",
          cond: {
            $gte: ["$$item.score", 70]
          }
        }
      }
    }
  }
])

```

Memory: Use \$\$ for variables.

Q8. Return subject names.

Concepts Used: \$map

```

db.students.aggregate([
  {
    $project: {
      name: 1,
      subjects: {
        $map: {
          input: "$marks",
          as: "item",
          in: "$$item.subject"
        }
      }
    }
  }
])

```

Memory: JS: map().

Q9. Return passed count.

Concepts Used: \$size • \$filter

```

db.students.aggregate([
  {
    $project: {
      name: 1,
      passedCount: {
        $size: {
          $filter: {
            input: "$marks",
            as: "item",
            cond: {
              $gte: ["$$item.score", 70]
            }
          }
        }
      }
    }
  }
])

```

Memory: JS: filter().length.

Q10. Push order values.

Concepts Used: \$group • \$push

```

db.orders.aggregate([
  {
    $group: {

```

```

        _id: "$customer",
        totalSpent: { $sum: "$amount" },
        orders: { $push: "$amount" }
    }
}
])

```

Memory: \$push keeps duplicates.

Q11. Monthly revenue.

Concepts Used: \$year • \$month • \$sort

```

db.orders.aggregate([
  {
    $group: {
      _id: {
        year: { $year: "$createdAt" },
        month: { $month: "$createdAt" }
      },
      revenue: { $sum: "$amount" }
    }
  },
  { $sort: { "_id.year": 1, "_id.month": 1 } }
])

```

Memory: Sort using _id.year.

Q12. First and latest salary.

Concepts Used: \$sort • \$first • \$last

```

db.salaryHistory.aggregate([
  { $sort: { updatedAt: 1 } },
  {
    $group: {
      _id: "$employee",
      firstSalary: { $first: "$salary" },
      latestSalary: { $last: "$salary" }
    }
  }
])

```

Memory: Always sort first.

Q13. Senior vs Junior.

Concepts Used: \$cond

```

db.employees.aggregate([
  {
    $project: {
      name: 1,
      salary: 1,
      grade: {
        $cond: {
          if: { $gte: ["$salary", 80000] },
          then: "Senior",
          else: "Junior"
        }
      }
    }
  }
])

```

Memory: Use ['\$field', value].

Q14. Handle null phone.

Concepts Used: \$ifNull

```

db.users.aggregate([
  {
    $project: {
      name: 1,
      phone: {
        $ifNull: ["$phone", "Not Available"]
      }
    }
  }
])

```

Memory: JS: ?? operator.

Q15. Total marks.

Concepts Used: \$reduce

```

db.students.aggregate([
  {

```

```

    $project: {
      name: 1,
      totalMarks: {
        $reduce: {
          input: "$marks",
          initialValue: 0,
          in: {
            $add: ["$$value", "$$this"]
          }
        }
      }
    }
  }
}
])

```

Memory: JS: `reduce()`.

Q16. Advanced \$lookup.

Concepts Used: `$lookup` • `let` • `pipeline` • `$expr`

```

db.users.aggregate([
  {
    $lookup: {
      from: "orders",
      let: {
        parentUserId: "$_id"
      },
      pipeline: [
        {
          $match: {
            $expr: {
              $and: [
                { $eq: ["$userId", "$$parentUserId"] },
                { $eq: ["$status", "Delivered"] }
              ]
            }
          }
        },
        ],
      as: "deliveredOrders"
    }
  }
])

```

Memory: JS: `users.map()+filter()`.

Q17. Push full documents.

Concepts Used: `$$ROOT`

```

db.orders.aggregate([
  {
    $group: {
      _id: "$customer",
      orders: { $push: "$$ROOT" }
    }
  }
])

```

Memory: JS: `{...obj}`.

Q18. Multi-metric report.

Concepts Used: `$avg` • `$max` • `$min`

```

db.orders.aggregate([
  {
    $group: {
      _id: "$customer",
      totalSpent: { $sum: "$amount" },
      avgOrder: { $avg: "$amount" },
      highestOrder: { $max: "$amount" },
      lowestOrder: { $min: "$amount" }
    }
  }
])

```

Memory: One pass.

Q19. Dashboard metrics.

Concepts Used: `$group` • `$cond`

```

db.orders.aggregate([
  {
    $group: {
      _id: null,
      totalOrders: { $sum: 1 },
      deliveredOrders: {

```

```

    $sum: {
      $cond: [
        { $eq: ["$status", "Delivered"] },
        1,
        0
      ]
    },
  },
  pendingOrders: {
    $sum: {
      $cond: [
        { $eq: ["$status", "Pending"] },
        1,
        0
      ]
    }
  }
}
}
}
})

```

Memory: Conditional aggregation.

Q20. Dashboard with \$facet.

Concepts Used: \$facet • \$arrayElemAt • \$ifNull

```

db.orders.aggregate([
  {
    $facet: {
      totalOrders: [
        {
          $group: {
            _id: null,
            count: { $sum: 1 }
          }
        }
      ]
    }
  }
])

```

Memory: Promise.all().

Q21. Running total.

Concepts Used: \$setWindowFields

```

db.orders.aggregate([
  {
    $setWindowFields: {
      sortBy: { date: 1 },
      output: {
        runningTotal: {
          $sum: "$amount",
          window: {
            documents: ["unbounded", "current"]
          }
        }
      }
    }
  }
])

```

Memory: Cumulative values.

Q22. Gold / Silver / Bronze.

Concepts Used: \$switch

```

db.employees.aggregate([
  {
    $project: {
      name: 1,
      grade: {
        $switch: {
          branches: [
            {
              case: { $gte: ["$salary", 100000] },
              then: "Gold"
            },
            {
              case: { $gte: ["$salary", 60000] },
              then: "Silver"
            }
          ],
          default: "Bronze"
        }
      }
    }
  }
])

```

```

    }
  }
})

```

Memory: Use for 3+ branches.

Q23. Salary bands.

Concepts Used: \$switch • \$group

```

db.employees.aggregate([
  {
    $project: {
      salaryBand: {
        $switch: {
          branches: [
            {
              case: { $lt: ["$salary", 50000] },
              then: "0-50k"
            },
            {
              case: { $lt: ["$salary", 100000] },
              then: "50k-100k"
            }
          ],
          default: "100k-200k"
        }
      }
    }
  },
  {
    $group: {
      _id: "$salaryBand",
      count: { $sum: 1 }
    }
  }
])

```

Memory: Mention \$bucket if asked.

Q24. Board analytics.

Concepts Used: \$group • \$cond

```

db.cards.aggregate([
  {
    $group: {
      _id: "$board",
      totalCards: { $sum: 1 },
      completedCards: {
        $sum: {
          $cond: [
            { $eq: ["$status", "Done"] },
            1,
            0
          ]
        }
      },
      pendingCards: {
        $sum: {
          $cond: [
            { $ne: ["$status", "Done"] },
            1,
            0
          ]
        }
      },
      completedStoryPoints: {
        $sum: {
          $cond: [
            { $eq: ["$status", "Done"] },
            "$storyPoints",
            0
          ]
        }
      }
    }
  }
])

```

Memory: Every calculated field needs an accumulator.

Performance & Final Revision

```

db.orders.aggregate([
  { $match: { customer: "Satish" } },
  {
    $group: {
      _id: "$customer",

```

```
        totalSpent: { $sum: "$amount" }
      }
    }
  ]}).explain("executionStats")
```

Look for IXSCAN instead of COLLSCAN. Use allowDiskUse:true only after optimizing.

Your Personalized Weak Spots

- Use db.collection.aggregate(), not db.aggregate().
- Use \$_id after \$group.
- Every calculated field inside \$group needs an accumulator.
- Sort grouped fields using _id.year.
- 'as' must be a string.
- Use \$arrayElemAt after \$facet.
- Nested \$cond for else-if or use \$switch.